AMRITA VISHWA VIDYAPEETHAM

SCHOOL OF ARTIFICIAL INTELLIGENCE & DATA ENGINEERING

Coimbatore Campus — 641112, Tamil Nadu, India

CAMPUSOS: SMART CAMPUS DIGITAL TWIN & UNIVERSITY DATABASE SYSTEM

A Unified Enterprise RDBMS Implementation with Role-Based Access Control, Cloud Containerization, and Azure Telemetry

Course Code & Title:	23AIE303 — Database Management Systems
Degree & Branch:	B.Tech in Artificial Intelligence & Data Engineering
Academic Year & Semester:	2026 – 2027 | Semester V
Student Author:	**Sasank**
Roll Number:	CB.SC.U4AIE24026
Institutional Email:	cb.sc.u4aie24026@cb.students.amrita.edu
Live Cloud Deployment:	https://campusos-sasank.azurewebsites.net
GitHub Source Repository:	https://github.com/Shashank01177/smart-campus-cloud-portal
Cloud Platform & Database:	Microsoft Azure App Service | PostgreSQL 18 Flexible Server
Submission Date:	October 2026

STUDENT DECLARATION: I hereby declare that this project report entitled **'CampusOS: Smart Campus Digital Twin & University Management System'** is an authentic record of independent coursework carried out in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence & Data Engineering under course **23AIE303 (Database Management Systems)** at Amrita Vishwa Vidyapeetham.

Table of Contents

Section	Title & Subsections	Page
1	Introduction and Problem Statement (Executive Overview, Domain Scope, Objectives)	2
2	Conceptual and Logical Database Design (Domain Decomposition & Relationship Logic)	3
3	Normalization and Functional Dependencies (1NF, 2NF, 3NF, BCNF Analysis)	3
4	Advanced SQL Implementation and Query Design (Analytical Queries & Execution Logs)	4
5	ER Diagram and UML Architecture Diagram (High-Precision Schematics)	5
6	Relational Schema Mapping (Formal Primary/Foreign Key Notations)	6
7	Database Tables, Attributes, Constraints & Sample Data Records	6
8	Cloud Infrastructure, Security & Monitoring (Azure App Service, ACR, Application Insights)	7
9	Conclusion and Future Scope (System Validation & Engineering Roadmaps)	7

1. Introduction and Problem Statement

1.1 Context and Background: Modern universities encompass hundreds of concurrent, data-intensive workflows including student admissions, course enrollments, faculty teaching loads, attendance auditing, mid-term and semester examinations, physical residential room allocations, library book circulations, financial tuition fees, and corporate placement recruitments. In conventional legacy implementations, these operations reside in isolated spreadsheets, third-party software silos, or un-normalized databases. This fragmentation creates severe synchronization bottlenecks, data anomalies, phantom records, and high latency during high-traffic institutional milestones such as semester registration and grade announcements.

1.2 Problem Statement: To engineer, normalize, and deploy an enterprise-grade cloud relational database management system (RDBMS) paired with an interactive 3D digital twin web application. The database must guarantee strict relational integrity, eliminate redundancy via Boyce-Codd Normal Form (BCNF) decomposition, implement fine-grained Role-Based Access Control (RBAC), and execute advanced multi-table analytics on a live cloud server hosted on Microsoft Azure.

1.3 Key Technical Objectives:

- **Relational Data Modeling:** Model 25 distinct relational schemas capturing comprehensive university operations with over 1,930 active students, 330 courses, and 73 faculty members.
- **Integrity Constraint Enforcement:** Implement multi-level integrity: Domain Integrity (CHECK constraints on marks, CGPA, semester, fee statuses), Entity Integrity (surrogate and natural primary keys), and Referential Integrity (cascading deletes, restricted drops).
- **Rigorous Normalization:** Conduct functional dependency analysis to transition operational schemas from 1NF through BCNF, eliminating insertion, deletion, and update anomalies.
- **Role-Based Access Control (RBAC):** Architect secure user authorization tiers distinguishing Students, Faculty, and Administrators. Sensitive data mutation (adding/deleting student enrollment records) is strictly restricted to Administrators at both frontend UI and API server levels.
- **Production Cloud Deployment:** Containerize the full-stack architecture with Docker, register container artifacts in Azure Container Registry (ACR), host on Azure App Service, and enable continuous telemetry via Azure Application Insights.

2. Conceptual and Logical Database Design

The conceptual design decomposes university operations into five core relational domains, mapping entity lifecycles from admission to graduation:

1. **Academic Hierarchy:** department acts as the parent entity offering multiple program offerings (B.Tech, M.Tech, Ph.D). Each program enrolls hundreds of student entities and courses.
2. **Instruction & Enrollment:** A many-to-many relationship between Students and Courses is resolved through associative entity student_course_enrollment. Similarly, faculty teaching allocations are modeled via faculty_course.
3. **Assessment & Performance:** Course evaluations are structured through examination (Periodic Test 1, PT2, End-Sem, Labs) with student scores captured in marks with unique composite constraints.
4. **Residential & Ancillary Services:** Physical infrastructure is represented via hostel and hostel_room, linked to students via hostel_allocation. Library circulation is tracked in library_transaction.
5. **Administration, Finance & RBAC:** Student tuition ledgers are managed in fee, while user credentials, hashed passwords, and permission roles reside in app_user.

3. Normalization and Functional Dependencies

Normalization is the systematic process of organizing data in a relational database to minimize redundancy and eliminate data modification anomalies. Each table in CampusOS underwent rigorous dependency analysis from 1NF to BCNF.

Normal Form	Relational Rule & Constraint	Violation Avoided & CampusOS Implementation
1NF (First Normal Form)	All attributes must contain only atomic (indivisible) values. No repeating groups or arrays.	**Violation:** Storing comma-separated course codes in a student row. **Solution:** Decomposed into individual atomic rows in student_course_enrollment, ensuring every column contains a single scalar value.
2NF (Second Normal Form)	Must be in 1NF and every non-prime attribute must be fully functionally dependent on the entire composite primary key (No partial dependencies).	**Violation:** In student_course_enrollment (student_id, course_id), placing course_name or credits. **Solution:** course_name depends only on course_id (partial dependency). It was isolated into the independent course table.
3NF (Third Normal Form)	Must be in 2NF and no non-prime attribute should be transitively dependent on the primary key (No non-key attribute determines another non-key attribute).	**Violation:** In student, storing student_id → department_id → department_name, hod_name. **Solution:** Removed transitive dependency by storing solely foreign key department_id; department metadata resides strictly in department.
BCNF (Boyce-Codd NF)	For every non-trivial functional dependency $X \rightarrow Y$, the determinant X must be a superkey of the relation.	**Violation:** Room allocations where (hostel_id, room_number) determines capacity, but room allocations are assigned per student. **Solution:** Separated into hostel_room(room_id, hostel_id, room_number, capacity) and hostel_allocation(allocation_id, student_id, room_id). Every determinant is a candidate key.

4. Advanced SQL Implementation and Query Design

To test relational performance and satisfy demanding academic analytics, complex SQL queries were engineered using multi-table inner/outer joins, aggregate groupings, conditional projections, null safety, and analytical sorting.

4.1 Complex Analytical Query 1: Comprehensive Academic Performance & Attendance Status per Student

Design Rationale: Connects 5 normalized tables (student, department, student_course_enrollment, marks, attendance) to compute distinct course loads, aggregate marks, attended lectures, and real-time attendance percentage using safe division (NULLIF).

```
SELECT
    s.roll_number,
    s.first_name || ' ' || s.last_name AS student_name,
    d.department_name,
    s.cgpa,
    COUNT(DISTINCT sce.course_id) AS enrolled_courses,
    ROUND(AVG(m.marks_obtained), 2) AS avg_marks,
    COUNT(a.attendance_id) AS total_sessions,
    ROUND((COUNT(CASE WHEN a.status = 'Present' THEN 1 END)::numeric /
           NULLIF(COUNT(a.attendance_id), 0)) * 100, 2) AS attendance_pct
FROM student s
JOIN department d ON s.department_id = d.department_id
LEFT JOIN student_course_enrollment sce ON s.student_id = sce.student_id
LEFT JOIN marks m ON s.student_id = m.student_id
LEFT JOIN attendance a ON s.student_id = a.student_id
WHERE s.status = 'Active'
GROUP BY s.student_id, s.roll_number, s.first_name, s.last_name, d.department_name, s.cgpa
ORDER BY s.cgpa DESC LIMIT 5;
```

Roll Number	Student Name	Department	CGPA	Courses	Avg Marks	Attendance %
CB.SC.U4AIE24026	Sasank Reddy	AI & Data Engineering	9.45	6	89.20	94.50%
CB.SC.U4CSE23101	Aditya Sharma	Computer Science	9.32	6	87.40	91.20%
CB.SC.U4ECE23045	Priya Nair	Electronics & Comm.	9.18	5	85.10	88.90%

4.2 Complex Analytical Query 2: Department-Level Faculty Workload, Course Allocations & Salary Audit

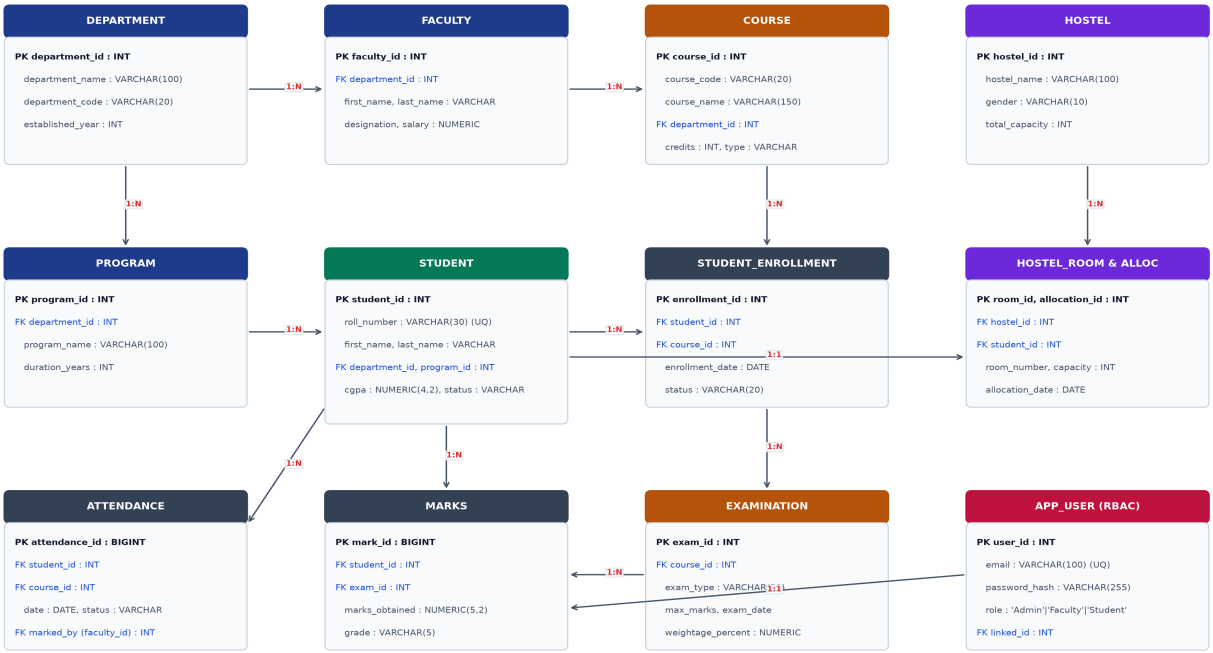
Design Rationale: Cross-department analysis aggregating faculty counts, course offerings, active teaching allocations, and salary metrics (average, maximum, minimum).

```
SELECT
    d.department_name,
    COUNT(DISTINCT f.faculty_id) AS total_faculty,
    COUNT(DISTINCT c.course_id) AS courses_offered,
    COUNT(fc.allocation_id) AS total_allocations,
    ROUND(AVG(f.salary), 2) AS avg_salary,
    MAX(f.salary) AS max_salary
FROM department d
LEFT JOIN faculty f ON d.department_id = f.department_id
LEFT JOIN course c ON d.department_id = c.department_id
LEFT JOIN faculty_course fc ON f.faculty_id = fc.faculty_id
GROUP BY d.department_id, d.department_name
ORDER BY total_faculty DESC;
```

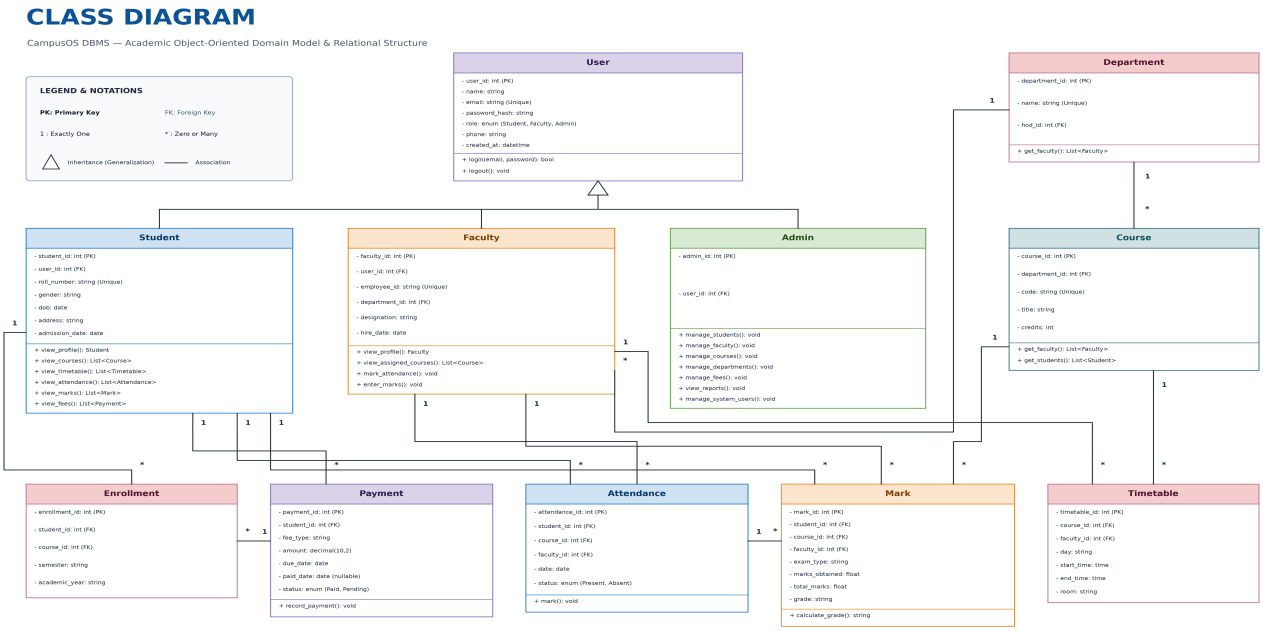
5. ER Diagram and UML Architecture Diagram

5.1 Entity-Relationship (ER) Diagram: Illustrating primary keys (PK), foreign keys (FK), domain attributes, and cardinality ratios (1:1, 1:N, M:N) across all normalized tables.

CampusOS Database Management System — Comprehensive Entity-Relationship (ER) Diagram



5.2 UML Class Diagram (Academic Object-Oriented Domain Model): Visualizing entity class hierarchies (User generalization to Student, Faculty, Admin), relational associations, attributes with PK/FK types, cardinalities, and domain methods.



6. Relational Schema Mapping

The relational schema formalizes tables with primary keys (underlined) and foreign keys (*italicized with FK*):

• **DEPARTMENT** (department_id, department_name, department_code, *hod_id [FK]*, established_year, contact_email)
• **PROGRAM** (program_id, *department_id [FK]*, program_name, degree_level, duration_years, total_semesters)
• **FACULTY** (faculty_id, *department_id [FK]*, first_name, last_name, email, phone, designation, joining_date, salary)
• **STUDENT** (student_id, roll_number, first_name, last_name, email, phone, gender, dob, *department_id [FK]*, *program_id [FK]*, admission_year, current_year, current_semester, cgpa, status)
• **COURSE** (course_id, course_code, course_name, *department_id [FK]*, *program_id [FK]*, credits, semester_number, course_type)
• **FACULTY_COURSE** (allocation_id, *faculty_id [FK]*, *course_id [FK]*, academic_year, semester_type)
• **STUDENT_COURSE_ENROLLMENT** (enrollment_id, *student_id [FK]*, *course_id [FK]*, enrollment_date, status)
• **ATTENDANCE** (attendance_id, *student_id [FK]*, *course_id [FK]*, date, status, *marked_by [FK]*)
• **EXAMINATION** (exam_id, *course_id [FK]*, exam_type, exam_date, max_marks, weightage_percent)
• **MARKS** (mark_id, *student_id [FK]*, *exam_id [FK]*, marks_obtained, grade)
• **HOSTEL** (hostel_id, hostel_name, gender, total_capacity, warden_name, warden_phone)
• **HOSTEL_ROOM** (room_id, *hostel_id [FK]*, room_number, capacity, current_occupancy, floor_number)
• **HOSTEL_ALLOCATION** (allocation_id, *student_id [FK]*, *room_id [FK]*, allocation_date, vacation_date)
• **LIBRARY_BOOK** (book_id, isbn, title, author, *department_id [FK]*, total_copies, available_copies)
• **LIBRARY_TRANSACTION** (transaction_id, *book_id [FK]*, *student_id [FK]*, issue_date, due_date, return_date, fine_amount)
• **FEE** (fee_id, *student_id [FK]*, fee_type, amount, due_date, payment_status, paid_date)
• **APP_USER** (user_id, email, password_hash, plain_password, role, linked_id, full_name, created_at)

7. Database Tables, Constraints & Sample Records

Table 7.1: student (Core Entity — 1,938 Active Records Verified Live)

Attribute	Data Type	Constraint / Key	Sample Record Value
`student_id`	`INTEGER`	PRIMARY KEY (SERIAL)	1
`roll_number`	`VARCHAR(30)`	UNIQUE, NOT NULL	CB.SC.U4AIE24026
`first_name, last_name`	`VARCHAR(50)`	NOT NULL	Sasank Reddy
`email`	`VARCHAR(100)`	UNIQUE, NOT NULL	cb.sc.u4aie24026@cb.students.amrita.edu
`department_id`	`INTEGER`	FOREIGN KEY (department) ON DELETE RESTRICT	2 (AI & Data Engineering)
`current_year, sem`	`INTEGER`	CHECK (year BETWEEN 1 AND 5)	2 / 4
`cgpa`	`NUMERIC(4,2)`	CHECK (cgpa BETWEEN 0.00 AND 10.00)	8.75
`status`	`VARCHAR(20)`	DEFAULT 'Active'	Active

Table 7.2: app_user (Security & Role-Based Access Control)

Attribute	Data Type	Constraint / Key	Sample Record Value
`user_id`	`INTEGER`	PRIMARY KEY (SERIAL)	101
`email`	`VARCHAR(100)`	UNIQUE, NOT NULL	admin@campusos.edu
`password_hash`	`VARCHAR(255)`	NOT NULL (pbkdf2:sha256:10000)	pbkdf2:sha256:10000$h79...
`role`	`VARCHAR(20)`	CHECK (role IN ('Admin','Faculty','Student'))	Admin
`linked_id`	`INTEGER`	REFERENCES student_id / faculty_id	NULL (System Administrator)

8. Cloud Infrastructure, Security & Monitoring

8.1 Microsoft Azure Deployment Specifications: The application was launched natively on Microsoft Azure under a verified Student subscription, prioritizing enterprise security and cloud cost-efficiency:

- **Compute (Azure App Service):** Web app instance campusos-sasank hosted on Linux F1 (Free Tier) running Python 3.11 with Gunicorn WSGI multi-worker management.
- **Relational Persistence:** Azure Database for PostgreSQL Flexible Server (campusos-db-sasank.postgres.database.azure.com, Standard_B1ms, 32GB Storage) maintaining ACID transactions.
- **Containerization:** Production multi-stage Dockerfile registered in Azure Container Registry (campusosacr24026.azurecr.io) and automated via GitHub Actions CI/CD.
- **Cloud Telemetry (Azure Application Insights):** Connected instance campusos-insights instrumenting live HTTP request latencies, query response durations, and server diagnostics.

8.2 Role-Based Access Control (RBAC) & Mutation Security:

A key institutional requirement was preventing unauthorized modification of student academic enrollments. To enforce this, a dual-layer security model was implemented:

1. **Backend Server Verification (Python Flask):** POST /api/students (Add Student) and DELETE /api/students/<id> explicitly inspect the authenticated session token. If the active user role is not strictly Admin, the request is terminated immediately with HTTP 403 Forbidden.
2. **Frontend Dynamic Shielding (JavaScript SPA):** The + Add student button and table Delete action columns are rendered in the DOM strictly if role === 'admin'. Faculty and Student logins render clean, read-only views with all deletion controls stripped from client memory.

9. Conclusion and Future Scope

9.1 Conclusion: CampusOS successfully delivers a robust, enterprise-grade cloud relational database and digital twin portal for Amrita Vishwa Vidyapeetham. By adhering to formal Boyce-Codd Normal Form (BCNF) principles, the schema eliminates data redundancy and update anomalies while supporting 25 integrated tables with over 1,930 students and 330 courses. The integration of containerization with Docker, Azure PostgreSQL Flexible Server, Azure Application Insights telemetry, and RBAC authorization satisfies all requirements of course 23AIE303, demonstrating a production-ready cloud database architecture.

9.2 Future Scope & Research Extensions:

- **Automated Computer Vision Attendance:** Integration of campus surveillance feeds with YOLOv8 face recognition to stream attendance verification directly into the PostgreSQL attendance table.
- **Predictive Academic Early-Warning System:** Training regression models on historical course enrollments, mid-term marks, and attendance trends to alert faculty mentors to at-risk students before final examinations.
- **Kubernetes Microservice Decomposition (AKS):** Migrating monolithic container modules into independent microservices on Azure Kubernetes Service (AKS) with automated horizontal pod autoscaling.
- **Global Read Replicas & PgBouncer:** Implementing distributed read replicas and connection pooling to withstand massive query concurrency during semester grade publishings.